OS LAB-9
DATE: 11/5/26

SUMANT SHRIDHAR
USN: 1BM24CS299
BATCH: 4K2

1) Banker's Algorithm

```
#include <stdio.h>

#define MAX 10

int n, m;
int alloc[MAX][MAX];
int max[MAX][MAX];
int need[MAX][MAX];
int available[MAX];

int isSafe() {
    int work[MAX];
    int finish[MAX] = {0};
    int count = 0, i, j, k;

    for (j = 0; j < m; j++) work[j] = available[j];

    while (count < n) {
        int found = 0;
        for (i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int possible = 1;
                for (j = 0; j < m; j++) {
                    if (need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }
                if (possible) {
                    for (k = 0; k < m; k++) {
                        work[k] += alloc[i][k];
                    }
                    finish[i] = 1;
                    count++;
                    found = 1;
                }
            }
        }
        if (!found) break;
    }
    return (count == n);
}

int main() {
```

```
int i, j;

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter number of resource types: ");
scanf("%d", &m);

printf("Enter Allocation Matrix:\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

printf("Enter Max Matrix:\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}

printf("Enter Available Resources:\n");
for (j = 0; j < m; j++) {
    scanf("%d", &available[j]);
}

for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

if (isSafe()) {
    printf("\nSystem is in SAFE state\n");
} else {
    printf("\nSystem is NOT in safe state\n");
}

int reqProcess;
int request[MAX];

printf("\nEnter process number making request (0-%d): ", n-1);
scanf("%d", &reqProcess);

printf("Enter request array:\n");
```

```
    for (j = 0; j < m; j++) {
        scanf("%d", &request[j]);
    }

    int valid = 1;
    for (j = 0; j < m; j++) {
        if (request[j] > need[reqProcess][j]) {
            valid = 0;
            break;
        }
    }
    if (!valid) {
        printf("\nError: Process has exceeded max claim.\n");
        return 0;
    }

    for (j = 0; j < m; j++) {
        if (request[j] > available[j]) {
            printf("\nResources not available. Process must wait.\n");
            return 0;
        }
    }

    for (j = 0; j < m; j++) {
        available[j] -= request[j];
        alloc[reqProcess][j] += request[j];
        need[reqProcess][j] -= request[j];
    }

    if (isSafe()) {
        printf("\nRequest can be granted. System remains in SAFE state.\n");
    } else {
        for (j = 0; j < m; j++) {
            available[j] += request[j];
            alloc[reqProcess][j] -= request[j];
            need[reqProcess][j] += request[j];
        }
        printf("\nRequest cannot be granted. System would be UNSAFE.\n");
    }

    return 0;
}
```

```
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:
0
1
0
2
0
0
3
0
2
2
1
1
0
0
2
Enter Max Matrix:
7
5
3
3
2
2
9
0
2
2
2
2
4
3
3
Enter Available Resources:
3
3
2

System is in SAFE state

Enter process number making request (0-4): 1
Enter request array:
1
0
2

Request can be granted. System remains in SAFE state.
```

Lab 9

11/5/26

Banker's Algorithm

```
#include <stdio.h>
#define MAX 10

int n, m;
int alloc[MAX][MAX], max[MAX][MAX], need
    [MAX][MAX], available[MAX][MAX];

int isSafe(){
  int work[MAX];
  int finish[MAX] = {0};
  int count = 0, i, j, k;

  for(j=0; j<m; j++) work[j] = available[j];

  while(count < n){
    int found = 0;
    for(i=0; i<n; i++){
      if(finish[i] == 0){
        int possible = 1;
        for(j=0; j<m; j++){
          if(need[i][j] > work[j]){
            possible = 0;
            break;
          }
        }
        if(possible){
          for(k=0; k<m; k++){
            work[k] += alloc[i][k];
          }
          finish[i] = 1;
          count++;
          found = 1;
        }
      }
    }
  }
```

```
        if(!found) break;
    }
    return (count == n);
}

int main(){
    int i, j;

    pf("Enter no of processes : ");
    sf("%d", &n);
    pf("Enter no of resource types : ");
    sf("%d", &m);
    pf("Enter Allocation Matrix : \n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            sf("%d", &alloc[i][j]);
        }
    }
    pf("Enter Mat Matrix : \n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            sf("%d", &max[i][j]);
        }
    }
    pf("Enter Available Resources : \n");
    for(j=0; j<m; j++){
        sf("%d", &available[j]);
    }
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}
```

```
if (isSafe) {
    printf("\nSystem is SAFE");
} else {
    pf("\nSystem is UNSAFE");
}

int reqP;
int req[MAX];

pf("Enter process no. requesting :");
sf("%d", &reqP);

pf("Enter request array : \n");
for(j = 0; j < m; j++){
    sf("%d", &req[j]);
}

int valid = 1;
for(j = 0; j < m; j++){
    if(req[j] > need[reqP][j]){
        valid = 0;
        break;
    }
}

if(!valid){
    pf("\nError: Process exceeded
        max claim.\n");
    return 0;
}

for(j = 0; j < m; j++){
    if(req[j] > available[j]){
        pf("\nResource not available.
            Process must wait.\n");
    return 0;
```

```
        }
    }
    for (j=0; j<m; j++){
        available[j] -= req[j];
        alloc[reqP][j] += req[j];
        need[reqP][j] -= req[j];
    }

    if (isSafe()){
        pf("\nRequest can be granted.
            System is safe.\n");
    }else{
        pf("\n Request cannot be granted.
            System would be unsafe.\n");
    }
    return 0;
}
```

— x —

O/p:

```
Enter no of processes: 5
Enter no of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
```

Enter Available Resources:
3 3 2

System is SAFE.

Enter process no. requesting: 1

Enter request array:
1 0 2

Request can be granted. System is safe.